

Sprawozdanie

Prowadzący: dr inż. Łukasz Maliński

Przedmiot	Semestr	Rok akademicki
Sieci komputerowe - projekt	IV	2025/2026

Skład sekcji:

Imię	Nazwisko
Konrad	Jaworek
Jakub	Sucheta

Spis treści

Spis treści.....	2
Wstęp i architektura systemu.....	2
Podział pracy nad projektem.....	2
Krótką historia pracy nad projektem.....	3
Opis działania protokołu komunikacyjnego, serializacji i wykrywania gubienia pakietów.....	3
Protokół komunikacyjny.....	3
Proces serializacji danych jest bezpośrednio uzależniony od typu wysyłanego pakietu.....	3
Pakiety typu 1 (konfiguracja).....	4
Pakiety typu 2 (sterowanie) oraz typu 3 (próbka symulacji).....	4
Ramka sterowania (typ 0x02).....	4
Ramka próbki symulacji (typ 0x03).....	4
Gubienie pakietów.....	5
Pomiar opóźnienia transportowego i redukcja jittera.....	5
Automatyczne wykrywanie instancji w sieci (Peer Discovery).....	5
Najistotniejsze napotkane trudności w realizacji tematu i sposobów ich rozwiązania ze szczególną uwagą poświęconą opisowi rozwiązań sieciowych.....	6
Zwięźle podsumowanie czego nauczył się każdy członek sekcji.....	6

Wstęp i architektura systemu

Celem projektu była implementacja rozproszonego układu sterowania w czasie dyskretnym, w którym pętla sprzężenia zwrotnego zamykana jest przez sieć komputerową. Zaimplementowana aplikacja działa w architekturze, w której jeden węzeł pełni rolę cyfrowego regulatora PID, a drugi symuluje obiekt z wykorzystaniem modelu ARX. Za warstwę komunikacyjną odpowiadają asynchroniczne klasy sieciowe frameworka Qt (QTcpSocket, QUdpSocket). Niezawodna i terminowa wymiana parametrów między węzłami (sygnał sterujący, wyjście obiektu) stanowi fundament stabilnego działania układu.

Podział pracy nad projektem

Konrad Jaworek	Jakub Sucheta
Walidacja projektu	Walidacja projektu
Naprawa błędów i uchybów otrzymanego projektu	Komunikacja sieciowa w CLI
Zmiany w gui (popup oraz przycisk połączenia)	Implementacja peer discovery przy użyciu UDP
Projektowanie budowy ramki protokołu	Projektowanie serializacji
Implementacja serializacji	Implementacja budowy ramki protokołu
Naprawa błędów i testowanie symulacji	Naprawa błędów i testowanie symulacji
Przygotowanie prezentacji	Przygotowanie prezentacji

Krótką historia pracy nad projektem

- Drobne poprawki otrzymanego projektu, dostosowanie interfejsu graficznego pod moduł sieciowy
- Zaprojektowanie podstawowego protokołu w trybie tekstowym (w CLI)
- Zrozumienie podstaw pracy komunikacji sieciowej w QT, realizacja podstawowego połączenia w cli
- Projekt wstępnego protokołu sieciowego (bardzo proste i trywialne przesyłanie danych)
- Ulepszenie protokołu, dodanie precyzyjniejszych komunikatów odnośnie połączenia (timeout, utrata pakietów, informacje odnośnie opóźnień)
- Znalazienie dziwnego błędu który zaburzał synchronizację wykresów po osi X
- Dodanie funkcjonalności w interfejsie użytkownika która informuje o zgubionych pakietach

Opis działania protokołu komunikacyjnego, serializacji i wykrywania gubienia pakietów

Protokół komunikacyjny

Budowa ramki protokołu

W zaimplementowanym programie zdefiniowano trzy główne typy wiadomości:

- konfigurację,
- sterowanie
- próbka symulacji (o tym poniżej w [Sposób serializacji danych...](#)).

Aby zapewnić spójność komunikacji, każda z wiadomości opiera się na ujednoliconym schemacie ramki, który składa się z nagłówka i pola danych. Nagłówek ma zawsze stały rozmiar wynoszący 5 bajtów i dzieli się na dwie sekcje. Pierwszy bajt określa typ przesyłanej wiadomości, jednoznacznie informując, czy pakiet zawiera parametry, sygnał sterujący, czy próbkę symulacji. Kolejne 4 bajty służą do zapisu rozmiaru przesyłanych danych, co pozwala systemowi przygotować się na odbiór odpowiedniej ilości informacji. Drugą częścią ramki jest właściwe pole Payload, którego całkowity rozmiar jest zmienny i zależy od typu danej wiadomości.

Nagłówek (5 bajtów):

- typ wiadomości (1 bajt : parametry / sterowanie / próbka symulacji)
- rozmiar danych (4 bajty)

Payload (zmienny rozmiar)

Proces serializacji danych jest bezpośrednio uzależniony od typu wysyłanego pakietu.

Pakiety typu 1 (konfiguracja)

Formatowane są tekstowo (format JSON).

Identyfikowane w nagłówku kodem 0x01, charakteryzują się zmiennym rozmiarem. Służą one do przesyłania struktury zawierającej niezbędne parametry symulacji, w tym parametry modelu ARX, nastawy regulatora PID oraz konfigurację generatora wartości zadanej.

Pakiety typu 2 (sterowanie) oraz typu 3 (próbka symulacji)

Formatowane są z użyciem payloadu binarnego i mają stałą długość.

Ramka sterowania (typ 0x02)

Ramka sterowania (typ 0x02) zajmuje dokładnie 55 bajtów. Przenosi ona kolejno: 1-bajtową zmienną określającą rolę nadawcy, 8-bajtowy numer kroku w formacie uint64_t, następnie pięć 8-bajtowych zmiennych typu double odpowiadających za sterowanie (u), wartość zadaną (w) oraz składowe regulatora (uP, uI, uD), a na końcu 1-bajtową bitmaskę z flagami sterowania.

Ramka próbki symulacji (typ 0x03)

Ramka próbki symulacji (typ 0x03) ma stały rozmiar 38 bajtów. Zawiera ona identyfikator roli nadawcy (gdzie 1 oznacza Regulator, a 2 - Obiekt), numer kroku w formacie uint64_t, czas symulacji w sekundach (double) oraz dwie zmienne typu double opisujące wartość zadaną (w) i wyjście obiektu (y). System przewiduje również wykorzystanie wspomnianej wcześniej bitmaski flag do przesyłania akcji sterujących (takich jak resetowanie składowych I oraz D, a także uruchamianie, zatrzymywanie i resetowanie samej symulacji). Zdecydowano się na zastosowanie uniwersalnej struktury tej ramki dla obu kierunków komunikacji. Mimo że w drodze powrotnej (od obiektu do regulatora) część przesyłanych informacji jest technicznie nadmiarowa, utrzymanie jednolitego formatu uznano za optymalne podejście projektowe. Ewentualne zmniejszenie rozmiaru pakietu o 8 bajtów nie uzasadniało wprowadzania dodatkowej złożoności architektonicznej w postaci dedykowanego typu ramki zwrotnej.

typ 1 (konfiguracja)	typ 2 (sterowanie)	typ 3 (próbka symulacji)																																							
Nagłówek: typ 0x01 rozmiar zmienny rozmiar <pre>{ "KONFIGURACJA": { "SYMULACJA": { ... }, "PARAMETRY ARX": { ... }, "PARAMETRY PID": { ... }, "GENERATOR WARTOSCI ZADANEJ": { ... } } }</pre>	Nagłówek: typ 0x02 rozmiar 55 bajtów PAYLOAD - BINARNY <table><tr><td>RolaNadawcy</td><td>1 byte</td><td>1 = Regulator</td></tr><tr><td>Krok</td><td>8 bytes</td><td>uint64_t</td></tr><tr><td>u (sterowanie)</td><td>8 bytes</td><td>double</td></tr><tr><td>w (wartość zadana)</td><td>8 bytes</td><td>double</td></tr><tr><td>uP (składowa P)</td><td>8 bytes</td><td>double</td></tr><tr><td>uI (składowa I)</td><td>8 bytes</td><td>double</td></tr><tr><td>uD (składowa D)</td><td>8 bytes</td><td>double</td></tr><tr><td>Flagi sterowania</td><td>1 byte</td><td>Bitmaska</td></tr></table> bit 000: Brak akcji bit 001: ResetI bit 010: ResetD bit 100: reset symulacji bit 1000: zatrzymaj sym bit 10000: uruchom sym	RolaNadawcy	1 byte	1 = Regulator	Krok	8 bytes	uint64_t	u (sterowanie)	8 bytes	double	w (wartość zadana)	8 bytes	double	uP (składowa P)	8 bytes	double	uI (składowa I)	8 bytes	double	uD (składowa D)	8 bytes	double	Flagi sterowania	1 byte	Bitmaska	Nagłówek: typ 0x03 rozmiar 38 bajtów PAYLOAD - BINARNY <table><tr><td>RolaNadawcy</td><td>1 byte</td><td>2 = Obiekt</td></tr><tr><td>Krok</td><td>8 bytes</td><td>uint64_t</td></tr><tr><td>t (czas)</td><td>8 bytes</td><td>double [s]</td></tr><tr><td>w (wartość z.)</td><td>8 bytes</td><td>double</td></tr><tr><td>y (wyjście obj)</td><td>8 bytes</td><td>double</td></tr></table>	RolaNadawcy	1 byte	2 = Obiekt	Krok	8 bytes	uint64_t	t (czas)	8 bytes	double [s]	w (wartość z.)	8 bytes	double	y (wyjście obj)	8 bytes	double
RolaNadawcy	1 byte	1 = Regulator																																							
Krok	8 bytes	uint64_t																																							
u (sterowanie)	8 bytes	double																																							
w (wartość zadana)	8 bytes	double																																							
uP (składowa P)	8 bytes	double																																							
uI (składowa I)	8 bytes	double																																							
uD (składowa D)	8 bytes	double																																							
Flagi sterowania	1 byte	Bitmaska																																							
RolaNadawcy	1 byte	2 = Obiekt																																							
Krok	8 bytes	uint64_t																																							
t (czas)	8 bytes	double [s]																																							
w (wartość z.)	8 bytes	double																																							
y (wyjście obj)	8 bytes	double																																							

Zdjęcie obrazujące schemat serializacji danych w ramkach.

Gubienie pakietów

Mechanizm wykrywania gubionych pakietów działa na poziomie logiki aplikacji, a nie na poziomie samego TCP. Każda ramka niesie licznik kroku, dzięki któremu odbiorca rozpoznaje, czy przyszła nowa próbka, czy tylko duplikat albo ramka opóźniona.

W trybie pracy sieciowej program porównuje numer aktualnie odebranej próbki z numerem ostatnio poprawnie przyjętej ramki. Jeżeli w oczekiwanym cyklu symulacji nie pojawi się nowa próbka, mechanizm uznaje to za utratę pakietu. W takiej sytuacji stosowany jest Zero-Order Hold, czyli utrzymywana jest ostatnia znana wartość wyjścia, a regulator PID zostaje chwilowo zamrożony, żeby układ nie reagował na brak danych.

Dodatkowo działa watchdog, który kontroluje, czy ramki napływają regularnie. Jeśli przez dłuższy czas nie ma nowych ramek, liczba błędnych ramek przekroczy dopuszczalny próg albo opóźnienie transmisji stanie się zbyt duże, program przechodzi w tryb awaryjnego rozłączenia i uznaje połączenie za niesprawne.

Pomiar opóźnienia transportowego i redukcja jittera

System na bieżąco monitoruje czas przesyłu ramek, porównując rzeczywisty czas nadejścia każdego pakietu z oczekiwanym, teoretycznym interwałem symulacji. Ze względu na specyfikę protokołów sieciowych, naturalnym zjawiskiem są chwilowe wahania opóźnień transmisji (tzw. jitter). Aby uodpornić układ sterowania na te fluktuacje i uniknąć fałszywych alarmów ze strony modułu watchdog, zaimplementowano buforujący filtr medianowy, działający jako przesuwne okno dla ostatnich kilkudziesięciu pomiarów. Dzięki temu program wylicza stabilne, odsumione opóźnienie sieciowe, a interwencja awaryjna (rozłączenie) następuje dopiero w przypadku długotrwałego i trwałego pogorszenia jakości łącza.

Automatyczne wykrywanie instancji w sieci (Peer Discovery)

Aby zoptymalizować proces zestawiania połączenia w sieci lokalnej (LAN), wprowadzono mechanizm automatycznego wykrywania instancji oparty na protokole UDP. Węzeł pełniący rolę serwera cyklicznie rozgłasza informacje o swoim adresie IP oraz nasłuchującym porcie TCP, wykorzystując datagramy typu Broadcast. Pozwala to instancji klienckiej na dynamiczne odnalezienie serwera i nawiązanie połączenia bez konieczności manualnej konfiguracji adresacji przez użytkownika.

Najistotniejsze napotkane trudności w realizacji tematu i sposobów ich rozwiązania ze szczególną uwagą poświęconą opisowi rozwiązań sieciowych

Jedną z napotkanych trudności była analiza działania programu w różnych warunkach sieciowych, z racji że w programowaniu komunikacji sieciowej raczej mieliśmy do czynienia tylko z wyższą warstwą (HTTP, REST), musieliśmy zgłębić tajniki testowania niskopoziomowych rozwiązań.

- Testowanie dwóch instancji na tej samej maszynie lokalnej zawsze działało perfekcyjnie, znaleźliśmy przydatny program który te warunki zaburzał (<https://github.com/jagt/clumsy>), program pozwala na wybór różnych sposobów degradacji połączenia internetowego, z pewnością możemy stwierdzić że przyniósł on nam najwięcej cennych danych odnośnie naszego rozwiązania sieciowego.
- Testowanie aplikacji w sieci LAN oraz VPN, z racji że typowe sieci LAN mają zazwyczaj doskonałe testowanie aplikacji zawsze przebiegało w nich pomyślnie, dlatego pochylił się nad połączeniem zdalnym za pośrednictwem VPN (<https://www.zerotier.com/>), w ten sposób również mogliśmy przetestować nasze rozwiązanie odnośnie znajdowania innych instancji w sieci (UDP Broadcast)

Zwięzłe podsumowanie czego nauczył się każdy członek sekcji

Konrad - dzięki temu projektowi nauczyłem się walidować poprawność komunikacji sieciowej, projektować i implementować własny protokół ramek oraz rozumieć, jak ważna jest poprawna serializacja danych przed wysłaniem ich przez sieć. Nauczyłem się też wykrywać i naprawiać błędy w działaniu komunikacji oraz opracowywać interfejs aplikacji tak, aby całość działała stabilnie.

Jakub - nauczyłem się, jak działa niskopoziomowa komunikacja sieciowa w c++, jak wykorzystać UDP do wykrywania serwera w sieci lokalnej oraz jak tworzyć i odtwarzać ramki po drugiej stronie połączenia. Nauczyłem się również skuteczniej testować działanie systemu i przygotowywać rozwiązanie tak, aby było bardziej odporne na błędy i problemy transmisji.